

Lab 9 - DLX Scan

Nate Herbst
A02307138
Nathan Walker
A02364124

Introduction

In this lab, we add two new scan instructions — GD (0x34) for signed decimal input and GDU (0x35) for unsigned decimal input — that read an integer typed by the user on a serial terminal and store it into a destination register. This requires a UART receiver, an ASCII-to-integer conversion state machine, clock domain crossing FIFOs, and pipeline stall logic to pause execution until the user finishes typing.

Procedure

New Opcodes and Assembler Changes

The new instructions were added to the instruction set:

Instruction	Opcode	Format	Description
GD Rd	0x34	Op[31:26] Rd[25:21] Unused[20:0]	Get signed decimal integer from keyboard
GDU Rd	0x35	Op[31:26] Rd[25:21] Unused[20:0]	Get unsigned decimal integer from keyboard

The assembler (`find\_labels.c` and `Assembler.c`) was updated to recognize these mnemonics. The opcodes array was extended from 52 to 54 entries, and a new encoding case was added that places the destination register Rd at bits [25:21], similar to how PCH/PD/PDU encode their source register.

The opcode package (`decode\_reg\_pkg.vhd`) was updated with two new constants:

```
constant OP_GD   : std_logic_vector(5 downto 0) := "110100"; -- 0x34
constant OP_GDU  : std_logic_vector(5 downto 0) := "110101"; -- 0x35
```

Assembly Program

The factorial program was restructured to:

1. Print "Welcome to the DLX factorial program!" with CR+LF newline
2. Print "Enter a number: " prompt
3. Execute `GDU R1` to read the user's input
4. Echo the input with `PDU R1`
5. Compute the factorial using the existing multiply-by-repeated-addition subroutine
6. Print "N! = RESULT" with the input and factorial result
7. Loop back to `J end` (infinite loop; press reset to restart)

Proper CR+LF line endings were added (PCH with ASCII 10 for LF and ASCII 13 for CR) to ensure correct terminal display.

New Module: RX_UART (RX_UART.vhd)

This module was reused from Lab 1. It is a UART receiver with 8x oversampling that converts a serial bit stream into 8-bit bytes. It runs on the 153.6 kHz clock (8 times the 19200 baud rate) generated by the PLL. The receiver uses majority voting on the middle 4 samples of each bit period to determine the bit value, providing noise immunity. When a complete valid byte is received (start bit = 0, 8 data bits, stop bit = 1), it pulses `wrreq` for one cycle with the byte on `data\_out`.

New Module: Multiplication (multiplication.vhd)

This is a Quartus LPM_MULT IP block configured to multiply a 32-bit input by the constant 10. It has 8 pipeline stages to meet timing at 50 MHz. The input `dataa` is 32 bits wide, `datab` is hardcoded to the 4-bit constant "1010" (decimal 10), and the output `result` is 36 bits wide to prevent truncation. This multiplier is used by the ascii_to_int FSM for the `n = n \* 10` accumulation step.

New Module: ascii_to_int (ascii_to_int.vhd)

This is the core new module for Lab 9. It is a finite state machine that converts a stream of ASCII digit characters into a 32-bit integer. It runs at 50 MHz on the processor clock domain.

Algorithm: The FSM implements the standard string-to-integer conversion: $n = n * 10 + (\text{digit} - 48)$. Characters are read from the RX FIFO (which uses show-ahead mode, so the data is valid on q whenever rdempty = '0'). The FSM processes each character:

- **Digits ('0'-'9'):** Accumulate using the LPM_MULT multiplier (8-cycle wait for pipeline)

- **Minus sign ('-')**: Sets a negative flag if no digits have been seen yet (for signed GD)
- **Newline (0x0A) or Carriage Return (0x0D)**: Terminates the integer, optionally negates if minus was seen, and asserts scan_ready
- **All other characters**: Ignored

The FSM has 6 states: idle, read_char, process_char, wait_mult, add_digit, and done. The conversion starts immediately when characters arrive in the RX FIFO — it does not wait for the processor to reach a GD/GDU instruction. This means the integer is typically already converted and waiting by the time the processor needs it.

The done state holds the result on scan_data with scan_ready = '1' until the processor asserts scan_rdreq to consume it, then resets and returns to idle for the next integer.

Pipeline Integration (DLX_Processor.vhd)

Three new ports were added to `DLX\_Processor`: `scan\_data` (32-bit input), `scan\_ready` (input), and `scan\_rdreq` (output).

The scan stall logic uses a registered latch:

```
process(clk)
begin
    if rising_edge(clk) then
        if rst = '1' then
            scan_stalling <= '0';
        elsif scan_stalling = '0' and
            (internal_instr(31 downto 26) = OP_GD or
             internal_instr(31 downto 26) = OP_GDU) then
            -- GDU just appeared in Fetch output — start stalling
            scan_stalling <= '1';
        elsif scan_stalling = '1' and scan_ready = '1' then
            -- User input arrived — release the stall
            scan_stalling <= '0';
        end if;
    end if;
end process;

scan_stall <= scan_stalling;
```

Three new ports were added to `DLX\_Processor`: `scan\_data` (32-bit input), `scan\_ready` (input), and `scan\_rdreq` (output).

The scan stall logic uses a registered latch:

```
-- Consume: pulse scan_rdreq when GD/GDU reaches Execute (ID/EX) with
data ready.
scan_rdreq <= '1' when (id_ex_opcode = OP_GD or id_ex_opcode = OP_GDU)
                    and scan_ready = '1'
                    and flush = '0'
                    else '0';

-- Stall gated by flush (no stall during flush)
stall <= (stall_raw and not flush) or fifo_full or scan_stall;
```

When GD/GDU appears in the Fetch output (`internal\_instr`), the registered `scan\_stalling` flag latches high. This freezes the pipeline (Fetch holds, Decode inserts NOP bubbles) until `scan\_ready` goes high, indicating the ascii_to_int FSM has a converted integer available.

The consume signal fires when GD/GDU reaches the Execute stage:

```
scan_rdreq <= '1' when (id_ex_opcode = OP_GD or id_ex_opcode = OP_GDU)
                    and scan_ready = '1'
                    and flush = '0'
                    else '0';
```

The hazard detection unit was also updated to mark GD/GDU as not reading any source registers (`uses\_rs1 <= '0'`), preventing spurious load-use stalls.

Execute Stage Changes (execute.vhd)

A `scan\_data` input port was added, and the ALU output register input is muxed:

```
-- Scan data injection: for GD/GDU, use scan_data instead of ALU output
alu_or_scan <= scan_data when (opcode = OP_GD or opcode = OP_GDU)
                    else ALU_out;
```

When a GD/GDU instruction passes through Execute, `scan\_data` is latched into the ALU result register instead of the ALU output. This value then flows through Memory and Write-back normally — Write-back sees it as an ALU result and writes it to the destination register Rd. No changes were needed to the Memory or Write-back stages.

Top-Level Wiring (Lab9_DLX_Scan.vhd)

The top-level entity was extended with the complete RX data path:

```
ARDUINO_IO(0) ← USB-TTL ← serial terminal (keyboard)
```

→ RX_UART (153.6 kHz, 8x oversampling)
→ dcfifo (8-bit, dual-clock: 153.6 kHz write / 50 MHz read)
→ ascii_to_int FSM (50 MHz)
→ scan_data / scan_ready / scan_rdreq → DLX_Processor

The existing FIFO component (Quartus dcfifo, 256-deep, 8-bit, show-ahead mode) was reused for the RX clock domain crossing. The RX_UART writes at 153.6 kHz and the ascii_to_int FSM reads at 50 MHz.

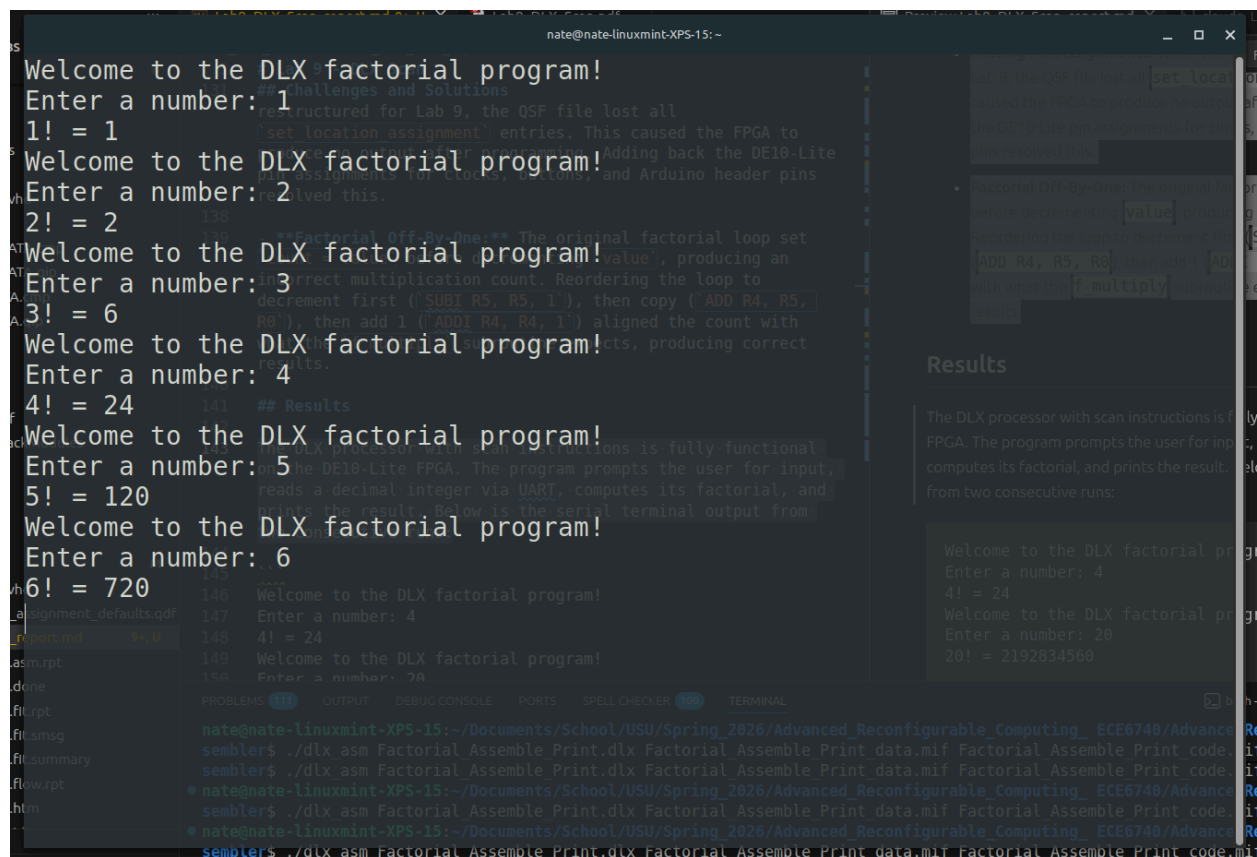
Challenges and Solutions

- **Show-Ahead FIFO Timing:** The RX dcfifo uses show-ahead mode, meaning data appears on `q` immediately when `rdempty = '0'`, before `rdreq` is asserted. Our initial `ascii_to_int` FSM asserted `rdreq` before reading the data, causing it to skip the first character. The fix was to read the character directly from `rx_char` in the `idle` state (since it's already valid), process it in `process_char`, and assert `rdreq` during processing to advance to the next character.
- **Pipeline Stall Timing:** The synchronous ROM in the Fetch stage introduces a 1-cycle latency — by the time GDU appears on `internal_instr`, the PC has already advanced past it. A simple combinational stall on `internal_instr` would only last 1 cycle before Decode clears it to NOP. The solution was a registered `scan_stalling` latch that stays high until `scan_ready` arrives, freezing the pipeline for as long as needed.
- **Missing Pin Assignments:** When the Quartus project was restructured for Lab 9, the QSF file lost all `set_location_assignment` entries. This caused the FPGA to produce no output after programming. Adding back the DE10-Lite pin assignments for clocks, buttons, and Arduino header pins resolved this.
- **Factorial Off-By-One:** The original factorial loop set `count = value` before decrementing `value`, producing an incorrect multiplication count. Reordering the loop to decrement first (`SUBI R5, R5, 1`), then copy (`ADD R4, R5, R0`), then add 1 (`ADDI R4, R4, 1`) aligned the count with what the `f_multiply` subroutine expects, producing correct results.

Results

The DLX processor with scan instructions is fully functional on the DE10-Lite FPGA. The program prompts the user for input, reads a decimal integer via UART, computes its factorial, and prints the result. Below is the serial terminal output from two consecutive runs. (See Figure 1)

Figures



```
WELCOME TO THE DLX FACTORIAL PROGRAM!
Enter a number: 1
1! = 1
WELCOME TO THE DLX FACTORIAL PROGRAM!
Enter a number: 2
2! = 2
WELCOME TO THE DLX FACTORIAL PROGRAM!
Enter a number: 3
3! = 6
WELCOME TO THE DLX FACTORIAL PROGRAM!
Enter a number: 4
4! = 24
WELCOME TO THE DLX FACTORIAL PROGRAM!
Enter a number: 5
5! = 120
WELCOME TO THE DLX FACTORIAL PROGRAM!
Enter a number: 6
6! = 720
```

Results

The DLX processor with scan instructions is fully functional on the DE10-Lite FPGA. The program prompts the user for input, reads a decimal integer via UART, computes its factorial, and prints the result. Below is the serial terminal output from two consecutive runs:

```
Welcome to the DLX factorial program!
Enter a number: 4
4! = 24
Welcome to the DLX factorial program!
Enter a number: 20
20! = 2192834560
```

Figure 1: Output results of Factorial program in 'screen' session.

Conclusion

In this lab, we successfully extended our custom DLX processor and assembler to support two new input instructions — GD (0x34) and GDU (0x35) — that read decimal integers from the keyboard via UART. The implementation required four new or reused hardware modules: the RX_UART receiver (from Lab 1), a dual-clock FIFO for crossing from the 153.6 kHz receive domain to the 50 MHz processor domain, an LPM_MULT IP core for multiply-by-10, and a custom ascii_to_int FSM that converts ASCII digit streams to 32-bit integers.

On the pipeline side, we added scan_data injection in the Execute stage (muxing scan_data onto the ALU result path for GD/GDU instructions) and a registered stall mechanism that freezes the pipeline until the user completes their input. The existing Write-back and Memory stages required no modification, as the scan data flows through the standard ALU result path.

The processor now supports full bidirectional UART communication: 5 I/O instructions (PCH, PD, PDU for output; GD, GDU for input) on top of the 49 original ALU, memory, branch, and jump instructions, for a total of 54 opcodes. The factorial demonstration program successfully reads user input, computes the factorial, and displays the result, meeting all pass-off requirements.

Appendix

Serial Output when running through our DLX processor

```
Welcome to the DLX factorial program!
Enter a number: 1
1! = 1
Welcome to the DLX factorial program!
Enter a number: 2
2! = 2
Welcome to the DLX factorial program!
Enter a number: 3
3! = 6
Welcome to the DLX factorial program!
Enter a number: 4
4! = 24
Welcome to the DLX factorial program!
Enter a number: 5
5! = 120
Welcome to the DLX factorial program!
Enter a number: 6
6! = 720
```

Lab9_DLX_Scan.vhd (top level file)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;

entity Lab9_DLX_Scan is
    port (

        -- Clock
        ADC_CLK_10 : in std_logic;
        MAX10_CLK1_50 : in std_logic;
        MAX10_CLK2_50 : in std_logic;
```



```

-- Tx      :   out std_logic;

-- Arduino Header
-- I/O 0 - Rx
-- I/O 1 - Tx

ARDUINO_IO      : inout std_logic_vector(15 downto 0);
ARDUINO_RESET_N : inout std_logic;

-- Push buttons (active low)
KEY : in std_logic_vector(1 downto 0)

);
end Lab9_DLX_Scan;

architecture component_list of Lab9_DLX_Scan is

    component FIFO
        PORT
        (
            data      : IN STD_LOGIC_VECTOR (7 DOWNTO 0);
            rdclk      : IN STD_LOGIC ;
            rdreq      : IN STD_LOGIC ;
            wrclk      : IN STD_LOGIC ;
            wrreq      : IN STD_LOGIC ;
            q          : OUT STD_LOGIC_VECTOR (7 DOWNTO 0);
            rdempty     : OUT STD_LOGIC
        );
    end COMPONENT;

    -- Component Declarations

    component PLL_UART
        PORT
        (
            inclk0      : IN STD_LOGIC := '0';
            c0          : OUT STD_LOGIC ;
            c1          : OUT STD_LOGIC
        );

```

```

end component;

component TX_UART
  port
  (
    Pclk : in std_logic;
    Tx: out std_logic;
    rdreq: out std_logic;
    empty: in std_logic;
    data_in: in std_logic_vector(7 downto 0)
  );
end component;

-- Signals
signal Rx : std_logic;
signal Tx : std_logic;

-- Clock Signals
signal clk_rx_8x : std_logic; -- 153.6 kHz (8 * 19200)
signal clk_tx_1x : std_logic; -- 19.2 kHz

-- RX path signals
signal rx_byte      : std_logic_vector(7 downto 0);
signal rx_wrreq     : std_logic;
signal rx_fifo_data : std_logic_vector(7 downto 0);
signal rx_fifo_empty : std_logic;
signal rx_fifo_rdreq : std_logic;

-- Scan (ascii_to_int -> DLX) signals
signal scan_data      : std_logic_vector(31 downto 0);
signal scan_ready     : std_logic;
signal scan_rdreq     : std_logic;

constant DATA_WIDTH : integer := 32;

-- TX path signals
signal tx_data_byte : std_logic_vector(7 downto 0);
signal tx_read_req : std_logic;
signal fifo_empty : std_logic;
signal fifo_full : std_logic;

```

```

signal fifo_wr          : std_logic;
signal fifo_data        : std_logic_vector(DATA_WIDTH-1 downto 0);
signal fifo_instr       : std_logic_vector(DATA_WIDTH-1 downto 0);

signal char_wr          : std_logic;
signal char             : std_logic_vector(7 downto 0);

begin

    ARDUINO_RESET_N <= 'Z';

    -- UART IO Assignments
    Rx <= ARDUINO_IO(0);          -- Read from IO0
    ARDUINO_IO(0) <= 'Z';        -- Tri-state IO0 so it can be used as
input
                                --
    ARDUINO_IO(1) <= Tx;          -- Write internal Tx to IO1

    ARDUINO_IO(15 downto 2) <= (others => 'Z'); -- Set unused pins to
high-Z

    -- PLL Instantiation
    pll_inst : PLL_UART
    PORT MAP (
        inclk0 => MAX10_CLK1_50,
        c0     => clk_rx_8x,
        c1     => clk_tx_1x
    );

    -- TX UART Instantiation
    tx_inst : TX_UART
    PORT MAP (
        Pclk    => clk_tx_1x,
        Tx      => Tx,
        rdreq   => tx_read_req,
        empty   => fifo_empty,
        data_in => tx_data_byte
    );

    character_fifo : FIFO

```

```
PORT MAP
```

```
(  
    data      => char,  
    rdclk     => clk_tx_1x,  
    rdreq     => tx_read_req,  
    wrclk     => MAX10_CLK1_50,  
    wrreq     => char_wr,  
    q         => tx_data_byte,  
    rdempty   => fifo_empty  
);
```

```
char_translator_inst : entity work.char_translator
```

```
port map (  
    clk => MAX10_CLK1_50,  
    fifo_wr => fifo_wr,  
    fifo_data => fifo_data,  
    fifo_instr => fifo_instr,  
  
    char => char,  
    char_wr => char_wr,  
    fifo_full => fifo_full  
);
```

```
-----  
-- RX UART CHAIN: ARDUINO_IO(0) -> RX_UART -> dcfifo -> ascii_to_int  
-----
```

```
-- RX UART: serial bits -> 8-bit bytes (runs at 153.6 kHz = 8x  
oversampling)
```

```
rx_inst : entity work.RX_UART  
port map (  
    clk      => clk_rx_8x,  
    Rx       => Rx,  
    wrreq    => rx_wrreq,  
    data_out => rx_byte  
);
```

```
-- RX FIFO: clock domain crossing 153.6 kHz -> 50 MHz (reuse existing
dcfifo)
```

```
rx_fifo_inst : FIFO
```

```
port map (
```

```
    data      => rx_byte,
    wrclk     => clk_rx_8x,
    wrreq     => rx_wrreq,
    rdclk     => MAX10_CLK1_50,
    rdreq     => rx_fifo_rdreq,
    q         => rx_fifo_data,
    rdempty   => rx_fifo_empty
```

```
);
```

```
-- ASCII-to-integer converter FSM (runs at 50 MHz)
```

```
ascii_to_int_inst : entity work.ascii_to_int
```

```
port map (
```

```
    clk        => MAX10_CLK1_50,
    rst        => not KEY(0),
    rx_char     => rx_fifo_data,
    rx_empty    => rx_fifo_empty,
    rx_rdreq    => rx_fifo_rdreq,
    scan_data   => scan_data,
    scan_ready  => scan_ready,
    scan_rdreq  => scan_rdreq
```

```
);
```

```
-----
-- DLX PROCESSOR
-----
```

```
processor_inst : entity work.DLX_Processor
```

```
generic map (
```

```
    WIDTH => 10,
    INSTR_WIDTH => 32
```

```
)
```

```
port map (
```

```

        clk => MAX10_CLK1_50,
        fifo_full => fifo_full,
        rst => not KEY(0),
        fifo_wr => fifo_wr,
        fifo_data => fifo_data,
        fifo_instr => fifo_instr,
        scan_data => scan_data,
        scan_ready => scan_ready,
        scan_rdreq => scan_rdreq
    );

end component_list;

```

ascii_to_int.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- ASCII-to-Integer Converter
--
-- Reads characters from an RX FIFO and converts decimal digit sequences
-- into 32-bit integers. Conversion begins immediately when characters
-- arrive (does not wait for a GD/GDU instruction).
--
-- Protocol:
--   - Digits '0'-'9' are accumulated: n = n * 10 + (digit - 48)
--   - A leading '-' sets the negative flag (for signed GD)
--   - Newline (0x0A) or carriage return (0x0D) terminates the integer
--   - All other characters are ignored
--   - Result is held on scan_data with scan_ready='1' until scan_rdreq
--     pulses to consume it
--
-- Uses the Quartus LPM_MULT IP (8 pipeline stages) for the
multiply-by-10.

entity ascii_to_int is
    port (
        clk          : in  std_logic;

```

```

    rst          : in  std_logic;

    -- RX FIFO interface (8-bit, read side at 50 MHz)
    rx_char       : in  std_logic_vector(7 downto 0);
    rx_empty      : in  std_logic;
    rx_rdreq      : out std_logic;

    -- DLX processor interface
    scan_data     : out std_logic_vector(31 downto 0);
    scan_ready    : out std_logic;
    scan_rdreq    : in  std_logic
);
end entity ascii_to_int;

architecture behavioral of ascii_to_int is

    type state_type is (idle, read_char, process_char, wait_mult,
add_digit, done);
    signal state : state_type := idle;

    signal n          : unsigned(31 downto 0) := (others => '0');
    signal is_negative : std_logic := '0';
    signal has_digits  : std_logic := '0';

    -- Saved digit value while waiting for multiplier
    signal saved_digit : unsigned(3 downto 0) := (others => '0');

    -- Multiplier pipeline counter (8 stages)
    signal mult_count  : integer range 0 to 8 := 0;

    -- Multiplier interface signals
    signal mult_in     : std_logic_vector(31 downto 0);
    signal mult_out    : std_logic_vector(35 downto 0);

    -- Character classification helpers
    signal char_val    : unsigned(3 downto 0);
    signal is_digit    : std_logic;
    signal is_newline  : std_logic;
    signal is_minus    : std_logic;

```

```

begin

    -- Combinational character classification
    char_val    <= unsigned(rx_char(3 downto 0));
    is_digit    <= '1' when rx_char >= x"30" and rx_char <= x"39" else '0';
    is_newline  <= '1' when rx_char = x"0A" or rx_char = x"0D" else '0';
    is_minus    <= '1' when rx_char = x"2D" else '0';

    -- Multiplier input is always n
    mult_in <= std_logic_vector(n);

    -- LPM_MULT instance: 32-bit * constant 10, 8 pipeline stages, 36-bit
result
    mult_inst : entity work.multiplication
        port map (
            clock    => clk,
            dataa    => mult_in,
            result   => mult_out
        );

    process (clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                state      <= idle;
                n           <= (others => '0');
                is_negative <= '0';
                has_digits  <= '0';
                saved_digit <= (others => '0');
                mult_count  <= 0;
                scan_ready  <= '0';
                rx_rdreq    <= '0';
            else

                -- Default: deassert one-cycle signals
                rx_rdreq <= '0';

                case state is

                    when idle =>

```



```

-- In show-ahead mode, rx_char is already valid
when
    -- rx_empty='0'. Go straight to process_char.
    if rx_empty = '0' then
        state <= process_char;
    end if;

    when read_char =>
        -- After consuming a character (rdreq pulse), wait
        -- one cycle for the FIFO to present the next value
        -- and update rdempty. Then return to idle.
        rx_rdreq <= '0';
        state <= idle;

    when process_char =>
        -- Consume this character from FIFO (advance to
next)

        rx_rdreq <= '1';
        if is_newline = '1' then
            -- End of integer input
            if has_digits = '1' then
                if is_negative = '1' then
                    scan_data <=
std_logic_vector(-signed(n));
                else
                    scan_data <= std_logic_vector(n);
                end if;
                scan_ready <= '1';
                state <= done;
            else
                -- Empty enter (e.g. stray CR after LF) -
ignore

                state <= read_char;
            end if;

        elsif is_digit = '1' then
            -- Save the digit, start waiting for n*10 from
multiplier.

            -- Multiplier input (mult_in = n) is already
being fed;

```

```

-- the result will be valid after 8 clock
cycles.

    saved_digit <= char_val;
    has_digits  <= '1';
    mult_count  <= 0;
    state       <= wait_mult;

elsif is_minus = '1' and has_digits = '0' then
    -- Leading minus sign
    is_negative <= '1';
    state       <= read_char;

else
    -- Ignore any other character
    state <= read_char;
end if;

when wait_mult =>
    -- Wait 8 clock cycles for multiplier pipeline
    if mult_count < 8 then
        mult_count <= mult_count + 1;
    else
        state <= add_digit;
    end if;

when add_digit =>
    --  $n = (n * 10) + \text{digit}$ 
    -- Take lower 32 bits of 36-bit multiply result
    n <= unsigned(mult_out(31 downto 0)) +
resize(saved_digit, 32);
    state <= idle;

when done =>
    -- Hold result until the processor consumes it
    if scan_rdreq = '1' then
        scan_ready <= '0';
        n <= (others => '0');
        is_negative <= '0';
        has_digits <= '0';
        state <= idle;
    end if;
end process;

```

```
        end if;  
  
        end case;  
    end if;  
end if;  
end process;  
end behavioral;
```